



Universidade de Brasília

Instituto de Ciências Exatas
Departamento de Ciência da Computação

Implementação e Análise de Complexidade do Algoritmo de Needleman-Wunsch

Luan A. Freitas & M.Sc. Raul C. de Souza

Grupo: Os Nomes Quase Invertidos

Relatório final da disciplina

Projeto de Complexidade e Análise de Algoritmos do Doutorado em Informática

Professor

Dr. Marcelo Antonio Marotta

Brasília

2026

Dedicatória

*Aos que acreditam que um bom algoritmo
pode fazer a diferença.*

*"O objetivo principal da educação é criar pessoas capazes de fazer coisas novas e não
simplesmente repetir o que as outras gerações fizeram." — Jean Piaget*

Agradecimentos

Agradecemos ao Prof. Dr. Marcelo Antonio Marotta pela condução da disciplina *Projeto de Complexidade e Análise de Algoritmos* e pelo suporte e oportunidades ao longo do desenvolvimento deste trabalho.

Resumo

Este trabalho aborda a análise de complexidade do **algoritmo de Needleman-Wunsch**, utilizado para resolver o problema de alinhamento global de sequências biológicas. São apresentadas e comparadas três variações do algoritmo implementadas em linguagem C: **recursão pura**, **memoização recursiva** (*Top-Down*) e **programação dinâmica iterativa** (*Bottom-Up*).

O esquema de pontuação adotado atribui $+1$ para *matches*, -1 para *mismatches* e -1 para *gaps*. A abordagem recursiva pura exibe complexidade exponencial $\mathcal{O}(3^{m+n})$, enquanto as versões com memoização e tabulação atingem complexidade de tempo $\mathcal{O}(m \times n)$ e espaço $\mathcal{O}(m \times n)$. A análise experimental demonstra que o *Bottom-Up* oferece o melhor desempenho prático, beneficiando-se da ausência de sobrecarga de chamadas recursivas e de padrões mais eficientes de acesso à memória.

Palavras-chave: Programação Dinâmica, Needleman-Wunsch, Alinhamento de Sequências, Bioinformática, Análise de Algoritmos

Abstract

This work presents a complexity analysis of the **Needleman-Wunsch algorithm**, used to solve the global biological sequence alignment problem. Three variations of the algorithm implemented in the C programming language are investigated and compared: **pure recursion**, **recursive memoization** (*Top-Down*), and **iterative dynamic programming** (*Bottom-Up*).

The scoring scheme assigns +1 for matches, -1 for mismatches, and -1 for gaps. The pure recursive approach exhibits exponential complexity $\mathcal{O}(3^{m+n})$, while the memoized and tabulated versions achieve $\mathcal{O}(m \times n)$ time and space complexity. Experimental analysis demonstrates that the Bottom-Up approach provides the best practical performance, benefiting from the absence of recursive-call overhead and more efficient memory access patterns.

Keywords: Dynamic Programming, Needleman-Wunsch, Sequence Alignment, Bioinformatics, Algorithm Analysis

Sumário

1	Introdução	1
1.1	Trabalhos Base	1
1.2	Objetivos	2
2	Implementação	3
2.1	Formulação do Problema	3
2.2	Recursão Pura	3
2.2.1	Análise de Complexidade — Recursão Pura	4
2.3	Top-Down com Memoização	4
2.3.1	Análise de Complexidade — Top-Down	5
2.4	Bottom-Up Iterativo	5
2.4.1	Análise de Complexidade — Bottom-Up	6
2.5	Análise Comparativa de Complexidade	6
2.6	Instância de Teste e Resultados	6
2.7	Métricas de Tempo de Resposta	7
3	Membros do Grupo e Atividades	9
3.1	Luan A. Freitas	9
3.2	Raul C. de Souza	9
4	Conclusão	10
4.1	Principais Dificuldades	10
4.2	Perspectivas Futuras	11
	Referências	12

Lista de Tabelas

2.1	Comparação de complexidade das três abordagens.	6
2.2	Matriz M para $S_1=\text{"ACTG"}$, $S_2=\text{"AATG"}$. Células em verde = <i>traceback</i>	7
2.3	Tempo de execução e pico de uso de RAM medidos pelo <code>monitor</code> . (†) Overhead do monitor domina; algoritmo <1 ms. (*) Estimativa extrapolada. (—) Execução bloqueada por limite de segurança.	8

Capítulo 1

Introdução

O alinhamento de sequências biológicas é uma tarefa fundamental em Bioinformática. Dado o DNA, RNA ou proteínas de dois organismos, identificar regiões similares permite inferir relações evolutivas, descobrir genes funcionais e apoiar o desenvolvimento de fármacos e diagnósticos genéticos.

O **alinhamento global** busca a correspondência ótima entre a totalidade de duas sequências S_1 e S_2 , admitindo substituições, inserções e deleções. A abordagem ingênua examinaria 3^{m+n} alinhamentos possíveis, tornando-se inviável para sequências reais. Recursão memorizada e Programação Dinâmica reduzem esse custo a $\mathcal{O}(m \times n)$, viabilizando a solução exata [1, 2].

O algoritmo de **Needleman-Wunsch (NW)**, proposto em 1970 [1], é o método clássico para alinhamento global de proteínas. Ele define uma matriz bidimensional M em que cada célula $M[i][j]$ representa o escore ótimo entre os prefixos de tamanho i de S_1 e tamanho j de S_2 , com penalidades para *gaps* (indels).

1.1 Trabalhos Base

Needleman & Wunsch (1970)

O artigo seminal [1] introduz o primeiro algoritmo de Programação Dinâmica para alinhamento global de proteínas. Os autores propõem uma matriz 2D em que cada célula $M[i][j]$ representa o escore do melhor alinhamento entre os prefixos de tamanho i e j , com fator de penalidade para *gaps* (indels), fundamentando toda uma área de alinhamento de sequências biológicas.

Sandes (2015) — CUDAlign

A tese [2] desenvolve o CUDAlign, implementação GPU paralela do algoritmo Smith-Waterman (variante local do NW). Atingiu 10,37 TCUPS com 384 GPUs NVIDIA Tesla M2090 por meio da arquitetura MASA e da otimização *Block Pruning*, que descarta blocos da matriz sem contribuição para o alinhamento ótimo, demonstrando a escalabilidade necessária para genomas reais.

1.2 Objetivos

Este trabalho implementa e compara três variações do algoritmo NW em linguagem C, sem bibliotecas externas, adotando o esquema de pontuação: $match = +1$, $mismatch = -1$, $gap = -1$. As abordagens analisadas são:

- **Recursão Pura** — tradução direta da recorrência, sem cache (complexidade exponencial $\mathcal{O}(3^{m+n})$);
- **Top-Down com Memoização** — recursão com tabela de cache ($\mathcal{O}(m \times n)$);
- **Bottom-Up Iterativo** — tabulação iterativa, abordagem mais eficiente na prática ($\mathcal{O}(m \times n)$, sem recursão).

Capítulo 2

Implementação

2.1 Formulação do Problema

Dadas $S_1 = s_1s_2 \cdots s_m$ (tamanho m) e $S_2 = t_1t_2 \cdots t_n$ (tamanho n), o problema consiste em encontrar o alinhamento global que maximiza o escore total. A relação de recorrência de Needleman-Wunsch é:

$$M[i][j] = \max \begin{cases} M[i-1][j-1] + \text{sc}(S_1[i], S_2[j]) & \text{(diagonal)} \\ M[i-1][j] + \text{gap} & \text{(acima)} \\ M[i][j-1] + \text{gap} & \text{(esquerda)} \end{cases} \quad (2.1)$$

onde $\text{sc}(a, b) = +1$ se $a = b$, e -1 caso contrário. A inicialização é $M[0][0] = 0$, $M[i][0] = i \cdot (-1)$ e $M[0][j] = j \cdot (-1)$. O escore ótimo encontra-se em $M[m][n]$ e o alinhamento é recuperado por *traceback*.

2.2 Recursão Pura

A tradução direta da Equação (2.1) sem armazenamento de resultados intermediários produz a implementação mais simples, porém mais ineficiente.

```
1 int recursao_pura(int i, int j) {
2     if (i == 0) return j * GAP;
3     if (j == 0) return i * GAP;
4     int d = recursao_pura(i-1, j-1) + sim(S1[i-1], S2[j-1]);
5     int u = recursao_pura(i-1, j) + GAP;
6     int l = recursao_pura(i, j-1) + GAP;
7     return max3(d, u, l);
8 }
```

Listagem 2.1: Recursão pura sem memoização.

O problema central é a **recomputação de subproblemas sobrepostos**: cada chamada a `recursao_pura(i, j)` ramifica em até três subchamadas sem verificar se aquele estado já foi calculado. Por exemplo, o estado `recursao_pura(i-2, j-1)` é alcançado por múltiplos caminhos distintos na árvore de recursão.

2.2.1 Análise de Complexidade — Recursão Pura

Tempo

A recorrência de custo satisfaz

$$T(m, n) = T(m-1, n-1) + T(m-1, n) + T(m, n-1) + \mathcal{O}(1),$$

com $T(0, n) = T(m, 0) = \mathcal{O}(1)$. A profundidade máxima da árvore é $m + n$; com fator de ramificação 3: $\mathcal{O}(3^{m+n})$ — **exponencial**. Para $m = n = 30$: $\approx 3^{60} \approx 4 \times 10^{28}$ operações — completamente inviável na prática.

Espaço

Apenas a pilha de chamadas ativas, com profundidade máxima $m + n$: $\mathcal{O}(m + n)$.

2.3 Top-Down com Memoização

A função verifica a tabela `memo[i][j]` antes de calcular. Se `memo[i][j] ≠ UNDEF`, retorna o valor cacheado; caso contrário, resolve os três subproblemas, armazena e retorna (*lazy evaluation*).

```
1 int recursao_memo(int i, int j) {
2     if (memo[i][j] != UNDEF) return memo[i][j];
3     if (i == 0) return memo[i][j] = j * GAP;
4     if (j == 0) return memo[i][j] = i * GAP;
5     int d = recursao_memo(i-1, j-1) + sim(S1[i-1], S2[j-1]);
6     int u = recursao_memo(i-1, j) + GAP;
7     int l = recursao_memo(i, j-1) + GAP;
8     return memo[i][j] = max3(d, u, l);
9 }
```

Listagem 2.2: Top-Down: memoização recursiva.

A técnica de memoização consiste em armazenar o resultado de cada subproblema na primeira vez em que é computado, de modo que chamadas recursivas subsequentes

sejam respondidas em tempo constante por consulta à tabela, reduzindo a complexidade de exponencial para polinomial [3].

2.3.1 Análise de Complexidade — Top-Down

Tempo

Cada subproblema é computado no máximo uma vez. Há $(m+1) \times (n+1)$ subproblemas, cada um resolvido em $\mathcal{O}(1)$: $\mathcal{O}(m \times n)$.

Espaço

Tabela `memo` de tamanho $(m+1) \times (n+1)$ mais pilha de profundidade $m+n$: $\mathcal{O}(m \times n)$. Risco de *stack overflow* para $m, n > 10^4$.

2.4 Bottom-Up Iterativo

Inicializa as bordas da matriz ($M[i][0]$ e $M[0][j]$) e preenche todas as células em ordem crescente de i e j com dois laços aninhados. Elimina o *overhead* de chamadas recursivas, acessa memória de forma *cache-friendly* e suporta sequências de qualquer tamanho sem restrições de pilha.

```
1 void nw_iterativo() {
2     for (int i = 0; i <= m; i++) M[i][0] = i * GAP;
3     for (int j = 0; j <= n; j++) M[0][j] = j * GAP;
4     for (int i = 1; i <= m; i++)
5         for (int j = 1; j <= n; j++) {
6             int d = M[i-1][j-1] + sim(S1[i-1], S2[j-1]);
7             int u = M[i-1][j] + GAP;
8             int l = M[i][j-1] + GAP;
9             M[i][j] = max3(d, u, l);
10        }
11 }
```

Listagem 2.3: Bottom-Up: tabulação iterativa.

A Programação Dinâmica resolve problemas com subestrutura ótima e sobreposição de subproblemas preenchendo uma tabela de forma *bottom-up*, garantindo que cada subproblema seja computado exatamente uma vez e seu resultado reutilizado nos subproblemas maiores que dele dependem [3].

2.4.1 Análise de Complexidade — Bottom-Up

Tempo

Dois laços aninhados; cada célula calculada exatamente uma vez em $\mathcal{O}(1)$. Melhor, médio e pior caso: $\mathcal{O}(m \times n)$.

Espaço

Apenas a matriz M de tamanho $(m+1) \times (n+1)$: $\mathcal{O}(m \times n)$. Sem uso adicional de pilha.

2.5 Análise Comparativa de Complexidade

A Tabela 2.1 compara as três abordagens segundo os principais critérios de análise [3].

Tabela 2.1: Comparação de complexidade das três abordagens.

Critério	Rec. Pura	Top-Down (Memo)	Bottom-Up
Tempo	$\mathcal{O}(3^{m+n})$ exp.	$\mathcal{O}(m \times n)$	$\mathcal{O}(m \times n)$
Espaço	$\mathcal{O}(m + n)$ pilha	$\mathcal{O}(m \times n) +$ pilha	$\mathcal{O}(m \times n)$
Subproblemas	Todos recomp.	<i>Lazy</i> (necessários)	Todos, ord. fixa
Seq. longas	Inviável ($m, n > 20$)	Risco <i>overflow</i>	Sem limitações
<i>Overhead</i>	Cham. + recomp.	Cham. recursivas	Índice direto
Uso	Didático	Estudo / protótipo	Produção

2.6 Instância de Teste e Resultados

Para $S_1 = \text{"ACTG"}$ ($m = 4$) e $S_2 = \text{"AATG"}$ ($n = 4$), a Tabela 2.2 exibe a matriz M preenchida pelo algoritmo NW. As células sombreadas em verde correspondem ao caminho de *traceback*.

Tabela 2.2: Matriz M para $S_1=\text{"ACTG"}$, $S_2=\text{"AATG"}$. Células em verde = *traceback*.

	–	A	A	T	G
–	0	–1	–2	–3	–4
A	–1	1	0	–1	–2
C	–2	0	0	–1	–2
T	–3	–1	–1	1	0
G	–4	–2	–2	0	2

O escore ótimo $M[4][4] = 2$. O *traceback* produz o alinhamento abaixo, com “|” para *match* e “×” para *mismatch*:

S_1 :	A	C	T	G
		×		
S_2 :	A	A	T	G

Escore: A–A (+1), C–A (–1), T–T (+1), G–G (+1) $\Rightarrow$ **total** = 2.

Ambas as implementações Bottom-Up e Top-Down produzem resultado idêntico; o Bottom-Up apresenta menor tempo de execução e nenhum risco de estouro de pilha.

2.7 Métricas de Tempo de Resposta

Os experimentos foram realizados em processador Intel Core i5, 8 GB de RAM, sob Linux, com GCC com otimização (-O3). O monitoramento foi feito pelo executável `monitor` (incluído no projeto), que amostra `VmRSS` via `/proc/{pid}/status` a cada $500\ \mu\text{s}$ e mede o tempo total com `clock_gettime(CLOCK_MONOTONIC)`.

Tabela 2.3: Tempo de execução e pico de uso de RAM medidos pelo `monitor`. (†) Overhead do monitor domina; algoritmo <1 ms. (*) Estimativa extrapolada. (—) Execução bloqueada por limite de segurança.

Tamanho ($m = n$)	Tempo de Execução			Pico de RAM (KB)		
	Rec. Pura	Top-Down	Bottom-Up	Rec. Pura	Top-Down	Bottom-Up
5	< 1 ms	< 1 ms	< 1 ms †	1740	940	420
10	$\sim 2,4$ s	< 1 ms	< 1 ms †	1744	1356	736
15	$\sim 6,9$ min	< 1 ms	< 1 ms †	1744	1212	1748
20	~ 45 min*	< 1 ms	< 1 ms †	—	1740	412
100	inviável	< 1 ms	~ 9 ms	—	1844	1840
500	inviável	~ 5 ms	~ 119 ms	—	3492	3460
1000	inviável	~ 17 ms	~ 341 ms	—	6652	6592

A Tabela 2.3 apresenta dois grupos de métricas coletadas simultaneamente pelo `monitor`: o tempo total de execução e o pico de memória RSS (*Resident Set Size*), que representa a maior quantidade de RAM física ocupada pelo processo em qualquer instante durante sua execução.

No eixo do tempo, os resultados confirmam a inviabilidade da Recursão Pura a partir de $m = n \approx 20$: já em $n = 10$ o tempo chega a $\sim 2,4$ s, e em $n = 15$ ultrapassa 6,9 min, evidenciando o crescimento exponencial $\mathcal{O}(3^{m+n})$. O Top-Down apresentou tempo menor que o Bottom-Up a partir de $n = 500$, reflexo do comportamento *lazy* da memoização: apenas os subproblemas necessários ao *traceback* são avaliados, evitando o preenchimento completo da matriz.

No eixo da memória, as abordagens revelam comportamentos distintos. A Recursão Pura mantém pico constante em $\sim 1,7$ MB, pois utiliza apenas a pilha de chamadas — memória $\mathcal{O}(m + n)$ — sem alocar estruturas adicionais. O Top-Down e o Bottom-Up crescem juntos de $\sim 1,8$ MB em $n = 100$ para $\sim 6,5$ MB em $n = 1000$, comportamento coerente com a alocação da matriz $\mathcal{O}(m \times n)$ compartilhada por ambas as abordagens.

Capítulo 3

Membros do Grupo e Atividades

Este capítulo descreve a participação de cada membro do grupo no desenvolvimento do trabalho.

3.1 Luan A. Freitas

Responsável pela pesquisa bibliográfica e revisão do trabalho base, incluindo o artigo seminal de Needleman & Wunsch (1970) e a tese de Sandes (2015) sobre o CUDAlign.

Implementou as três variações do algoritmo em linguagem C:

- Recursão pura (Listagem 2.1);
- Memoização recursiva Top-Down (Listagem 2.2);
- Tabulação iterativa Bottom-Up (Listagem 2.3).

Desenvolveu a função de *traceback* para recuperação do alinhamento ótimo.

3.2 Raul C. de Souza

Responsável pela verificação e entendimento dos experimentos de desempenho, verificação e análise das métricas de tempo de execução para diferentes tamanhos de entrada (Tabela 2.3), e elaboração das sínteses conclusivas, relatório e apresentação.

Contribuiu na discussão e escrita das seções de Introdução, Formulação do Problema, Análise de Complexidade e Conclusão do relatório. Realizou a análise formal de complexidade de tempo e espaço(memória) para cada abordagem. Também elaborou os diagramas e tabelas comparativas presentes neste relatório

Realizou a revisão geral do código-fonte e do relatório final, além de preparar o pacote de entrega (.zip) para submissão no Moodle.

Capítulo 4

Conclusão

Este trabalho implementou e analisou três variações do algoritmo de Needleman-Wunsch para alinhamento global de sequências biológicas: Recursão Pura, Memoização Top-Down e tabulação Bottom-Up iterativa. Os resultados teóricos e experimentais confirmam a evolução natural no projeto de algoritmos:

- A **Recursão Pura** é $\mathcal{O}(3^{m+n})$ — exponencial e inviável para sequências reais a partir de $m, n \approx 20$;
- A **Memoização Top-Down** reduz para $\mathcal{O}(m \times n)$ ao evitar recomputações, mas mantém *overhead* de pilha e risco de *stack overflow*;
- O **Bottom-Up Iterativo** atinge $\mathcal{O}(m \times n)$ sem recursão, com acesso *cache-friendly* e sem limitações de tamanho — é a solução preferida para uso prático.

4.1 Principais Dificuldades

A principal dificuldade foi formalizar a análise de complexidade da recursão pura: demonstrar que a recorrência

$$T(m, n) = T(m-1, n-1) + T(m-1, n) + T(m, n-1) + \mathcal{O}(1)$$

resulta em $\mathcal{O}(3^{m+n})$ exigiu o uso de árvores de recursão e a argumentação sobre o fator de ramificação máximo em cada nível.

Outro desafio foi a implementação correta do *traceback*, especialmente nos casos de empate entre as três direções (diagonal, acima, esquerda), onde a escolha da direção preferencial afeta o alinhamento retornado, mas não o escore ótimo.

4.2 Perspectivas Futuras

Para sequências genômicas reais, com bilhões de pares de bases, técnicas avançadas como paralelismo em GPU (CUDAAlign [2]) tornam-se indispensáveis. O estudo evidenciou que a análise de complexidade vai além da notação assintótica: fatores de constante, padrões de acesso à memória e *overhead* de sistema têm impacto significativo no desempenho prático, especialmente em domínios de alto desempenho como a Bioinformática.

Em síntese, o trabalho reforçou a importância da Programação Dinâmica como técnica de projeto de algoritmos e forneceu experiência prática na implementação e avaliação de algoritmos clássicos de alinhamento de sequências.

Referências

- [1] Needleman, Saul B. e Christian D. Wunsch: *A general method applicable to the search for similarities in the amino acid sequence of two proteins*. Journal of Molecular Biology, 48(3):443–453, 1970. 1
- [2] Oliveira Sandes, Edans Fábio de: *Algoritmos paralelos exatos e otimizações para alinhamento de sequências biológicas longas em plataformas de alto desempenho*. Tese de Doutorado, Universidade de Brasília, Brasília, Brasil, 2015. 1, 2, 11
- [3] Cormen, Thomas H., Charles E. Leiserson, Ronald L. Rivest e Clifford Stein: *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edição, 2009. 5, 6